1 - Authentication & Creating Users

Now that we have full CRUD functionality, we're going to start on authentication and user registeration. Once we do that, in the next section we can start adding ownership and authorization.

There's many ways to implement authentication in Laravel. We're going to be using the tools that are given such as the Auth facade, helper and directive, but I'll also show you how you can use Laravel Breeze to scaffold up a complete system with views and forms. We're going to go over how sessions work and the session helper. Well add the register and login forms and hook them up and we'll create a logout and show specific items in the header based on our authentication status.

2 - Authentication Methods For Laravel

There are a ton of ways to implement authentication in Laravel. I just want to touch on some of the most common ones.

Custom Authentication

Laravel makes it easy to build your own custom authentication system. You can use the `make:auth` command to scaffold the authentication views and routes. You can also use the `auth` middleware to protect routes. You can use the `Auth` facade to authenticate users. You can use the `Hash` facade to hash passwords. You can use the `bcrypt` helper function to hash.

Laravel Breeze

Laravel Breeze is a minimalistic authentication starter kit that is included with Laravel. It is a great way to get started with authentication. It includes login, registration, password reset, email verification, and two-factor authentication. Laravel is a "batteries included" framework. It includes a ton of features that you can use right out of the box. Breeze is one of those features.

Laravel Jetstream

Laravel Jetstream is a more feature-rich authentication starter kit that includes everything that Breeze has plus team management, API support, and more. Jetstream is built with Livewire and Inertia. Livewire is a full-stack framework for Laravel that makes building dynamic interfaces simple. Inertia is a library that allows you to build single-page applications using classic server-side routing and controllers. This is not something I would recommend for a small project for beginners. It is a great way to get started with Laravel if you are looking to build a larger application.

Laravel Fortify

Laravel Fortify is a frontend agnostic authentication backend for Laravel. Fortify powers the registration, authentication, and two-factor authentication features of Laravel Jetstream.

Laravel Sanctum

Laravel Sanctum is a lightweight package for API token authentication. Sanctum provides a simple way to authenticate single-page applications (SPAs) or mobile applications. Sanctum is great for building APIs that will be consumed by a frontend application built with React, Vue or another frontend framework.

Soclialite

Laravel Socialite is an optional package that allows you to authenticate with OAuth providers like Facebook, Twitter, Google, and GitHub. Socialite is a great way to allow users to authenticate with your application using their existing social media accounts.

What Are We Using?

I was a little confused on what to use for this project. Initially, I thought Breeze may be the way to go, however, as awesome as Breeze is for productivity, I don't think it's good for a course because you don't really understand what happen under the hood. It creates everything for you including routes, controllers, views, and models. So if you do use Breeze, you should use it at the very beginning of your project because it scaffolds everything for you.

Instead of using a starter kit like breeze, we'll build or own custom authentication with the helpers that Laravel gives us. This will reinforce our understanding of the framework and working with the MVC pattern.

With that said, I think it would still be a good idea to just show you how to get setup with Breeze. So we'll do that in the next lesson, which will be separate from our main project.

3 - Laravel Breeze Demo

As I explained in the last lesson, we're going to build our own authentication system. However, I still want to show you how to scaffold a project with Breeze. This will give you a good idea of what Breeze does for you. It's a great way to get started with authentication.

What Is Included With Breeze?

Breeze includes routes, controllers, models and views with styled working forms for:

- Login
- Registration
- Password Reset
- Email Verification
- Two-Factor Authentication

New Laravel Project

DO NOT do any of this in our current Workpoia project. Open your terminal in a completely new folder. Run the following command to create a new Laravel project:

```
composer create-project laravel/laravel breeze-demo
cd breeze-demo
```

Install Breeze

Run the following command to install Breeze:

```
composer require laravel/breeze --dev
```

Now we can run the following command to setup the controllers, routes, views and other resources:

```
php artisan breeze:install
```

You will then be asked which Breeze stack do you want to use. These are the options:

- Blade with Alpine
- Livewire (Volt Class API) with Alpine
- Livewire (Volt Functional API) with Alpine
- React with Inertia
- Vue with Inertia
- API only

This shows you how many different types of applications that you can build with Laravel. We're going to use the default option and type in `blade` and hit enter.

I will say no for dark mode.

Just hit enter when it asks about tests.

Run The Server

Now you can run the following command to start the server. I will use a different port since I am still running the Workpoia project on port 8000.:

```
php artisan serve --port 8001
```

Now you can visit `http://localhost:8001` and you should see a login and register link. You can click on the register link and create a new user. You will then be redirected to the dashboard. There is also a profile page that you can visit.

How insane is that? You have a fully functioning authentication system that you created in literally seconds. This is the power of Laravel.

I am going to stop the server and delete the project. I just wanted to show you the awesomeness. If you want to inspect the code and see the routes, controllers, etc, you can do that.

I am going to get back to our project and create a custom authentication system.

4 - How Sessions & Authentication Work in Laravel

Before we write any code, I want to talk a little bit about how sessions work in Laravel. I don't want you to just learn the syntax and not understand what is actually happening under the hood. We will also use the `session` helper function to manually create session data.

Sessions

A session is a way to store information across multiple requests in your application. HTTP on it's own is a stateless protocol. This means that each request is independent of the previous request. This is great for performance, but it can be a problem when you need to maintain state information across multiple requests.

In Laravel and in many other frameworks and web apps, sessions are used to maintain state information, such as user authentication status, flash messages, etc. We've already seen an example of flash messages.

Authentication & Sessions

When a user logs in, Laravel will create a session for that user and store the user's authentication status in the session. When the user logs out, Laravel will destroy the session.

If we submit a new job listing, ultimately we want to get the user ID from the session and store it in the database with the listing. Then when we have functionality like updating and deleting listings, we want to check if the user is logged in and if they are the owner of the listing.

Session Cookies

As I saids, when a user logs in, Laravel will create a session for that user and store the user's authentication status in the session. This session is stored in a cookie on the user's browser. The cookie contains the session ID, which is a unique identifier for the session. The session ID is sent to the server with each request. The server uses the session ID to retrieve the session data from the database.

By default, Laravel uses a cookie named laravel_session or your_project_name_session to store the session ID. You can open your browser's developer tools and see the cookie. The data will be encrypted and signed to prevent tampering. Your app has a key defined in the `.env` file that is used to encrypt and sign the data. It has the key `APP_KEY` and the value is a random string of characters.

Remember Me Cookie

If the "Remember Me" option is enabled during login, Laravel will generate a long-lived cookie called remember_token. This token is stored in the remember_token field of the users table. If the session expires (e.g., the browser is closed), Laravel can automatically log the user back in using this cookie the next time they visit the site.

CSRF Tokens

Laravel also uses a CSRF token to prevent cross-site request forgery attacks. This token is stored in the session and is sent to the server with each request. The server uses the token to verify that the request is legitimate.

Session Configuration & Database Setup

Sessions are configured in the `config/session.php` file. Laravel supports different session drivers such as file, cookie, database, memcached, redis, and array. We are using the `database` driver in this project.

If you look in your database using PG Admin or something else, you will see a table called `sessions`. This is where Laravel stores the session data. There is a field called `payload` that contains the session data. This data is serialized and encrypted before being stored in the database. This data could be anything from user authentication status to CSRF tokens to flash messages.

There is also a `user_id` field that is used to associate the session with a user. This is used to determine if the user is logged in or not. Right now, any sessions you have will not have a user_id because we have not logged in yet.

`session` Helper Function

We can manually create session data. To do this, we can use the `session` helper function.

Let's do this in our `HomeController`. It doesn't matter where you put this code because we aren't keeping it, but I will put it in the `index()` method.

```
session()->put('test', '123');
$value = session()->get('test');
dd($value);
```

Now when you go to the homepage, you should see `123`. This is how you can manually create session data.

If you look in the database, you this data will be in the session table but it will be encrypted. Also, a single record can have multiple values. You can copy the payload and paste it into a site like https://www.base64decode.org/ or even ChatGPT to see what the data looks like.

You can now delete the session data by calling the `forget()` method.

```
session()->put('hello', 'world');
session()->put('test', '123');
session()->forget('test');
$value = session()->get('test'); // This will return 'world'
dd($value);
```

Now you will see `null` because the `test` key has been forgotten.

Delete that code.

So that is how sessions work in Laravel.

5 - Login & Register Controllers

We are going to build a custom login using some of the tools that Laravel offers. We even have the tables already created for us. We have a users table from the default migration files that come with Laravel. We also

have a `User` model that is already setup for us.

Create Controllers

When it comes to structure, there are many ways that you can handle authentication. A common approach is to create a controller for each type of authentication. For example, you might have a `LoginController` and a `RegisterController`. This allows you to keep your code organized and easy to maintain.

Let's create two new controllers:

```
php artisan make:controller LoginController
php artisan make:controller RegisterController
```

This will create two new files in the `app/Http/Controllers` directory. You can find them at `app/Http/Controllers/LoginController.php` and `app/Http/Controllers/RegisterController.php`.

Create Routes

Next, we need to create routes for our new controllers. Open the `routes/web.php` file.

We need to import the `LoginController` and `RegisterController` classes. Add the following to the top of the file.

```
use App\Http\Controllers\RegisterController;
use App\Http\Controllers\LoginController;
```

Then add the following routes.

```
Route::get('/register', [RegisterController::class, 'register']);
Route::post('/register', [RegisterController::class, 'store']);
Route::get('/login', [LoginController::class, 'login']);
Route::post('/login', [LoginController::class, 'authenticate']);
```

The first route is for the registration form. The second route is for the registration form submission. The third route is for the login form. The fourth route is for the login form submission.

Naming Routes

You can also apply a name to the routes. This is useful for generating URLs.

Let's give all of the routes a name except for the resource routes.

```
Route::get('/', [HomeController::class, 'index'])->name('home');
Route::get('/jobs/{id}/save', [JobController::class, 'save'])->name('jobs.save');
Route::resource('jobs', JobController::class);
```

```
Route::get('/register', [RegisterController::class, 'register'])-
>name('register');
Route::post('/register', [RegisterController::class, 'store'])-
>name('register.store');
Route::get('/login', [LoginController::class, 'login'])->name('login');
Route::post('/login', [LoginController::class, 'authenticate'])-
>name('login.authenticate');
```

Register Controller

Let's start with the `RegisterController`. Add the following imports for the types:

```
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;
```

We will create a `register` method that will return the registration form. We will also create a `store` method that will handle the registration form submission.

Let's start with the `register` method.

```
// @desc  Show register form
// @route GET /register
public function register(): View {
  return view('auth.register');
}
```

For now we will just create the view with a a heading or something. Create a folder called `auth` in the `resources/views` folder. Inside the `auth` folder create a file called `register.blade.php`.

```
<x-layout>
  <h1>Register</h1>
</x-layout>
```

Now you should be able to go to `http://localhost:8000/register` and see the heading.

Let's also create a method called `store` that will handle the registration form submission. For now, we will just return a string.

```
// @desc  Store new user
// @route POST /register
public function store(): string {
  return 'store';
}
```

We will come back to that later.

Login Controller

Let's add to the LoginController. Add the following imports for the types:

```
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;
```

We are going to create a few methods in the `LoginController`. We will create a `login` method that will return the login form. We will also create an `authenticate` method that will handle the login form submission.

Let's start with the `login` method.

```
// @desc  Show login form
// @route GET /login
public function login(): View {
  return view('auth.login');
}
```

In the `/resources/views/auth` folder, create a file called `login.blade.php` and add the following for now:

```
<x-layout>
  <h1>Login</h1>
</x-layout>
```

Now you should be able to go to `http://localhost:8000/login` and see the heading.

Create the `authenticate` method.

```
// @desc  Log in user
// @route POST /authenticate
public function authenticate(Request $request): string {
  return 'authenticate';
}
```

We will come back to that later.

6 - Register New User

We have our `RegisterController` and `register` method that returns a view. Let's add the form to that view. We can use our input components that we used in our other forms.

Open the `resources/views/auth/register.blade.php` file and add the following:

```
<x-layout>
  <div
    class="bg-white rounded-lg shadow-md w-full md:max-w-xl mx-auto mt-12 p-8 py-
12"
  >
    <h2 class="text-4xl text-center font-bold mb-4">Register</h2>
    <form method="POST" action="{{ route('register.store') }}">
      @csrf
      <x-inputs.text id="name" name="name" placeholder="Full name" />
      <x-inputs.text
        id="email"
        name="email"
        type="email"
        placeholder="Email address"
      />
      <x-inputs.text
        id="password"
        name="password"
        type="password"
        placeholder="Password"
      />
      <x-inputs.text
        id="password_confirmation"
        name="password_confirmation"
        type="password"
        placeholder="Confirm Password"
      />
      <button
        type="submit"
        class="w-full bg-blue-500 hover:bg-blue-600 text-white px-4 py-2 rounded
focus:outline-none"
      >
        Register
      </button>

      <p class="mt-4 text-gray-500">
        Already have an account?
        <a class="text-blue-900" href="{{route('login')}}">Login</a>
      </p>
    </form>
  </div>
</x-layout>
```

We added the input components, we are using the `@csrf` directive to add the CSRF token to the form and we are using the `route` helper to generate the URL for the `action` attribute as well as the link to the login page.

Notice that I used an underscore for `password_confirmation` It has to be named this if you want to use Laravels built-in validation rule for password confirmation.

`store` Method

Now let's handle the `store` method in our `RegisterController`. Open the `app/Http/Controllers/Auth/RegisterController.php` file and add the following imports:

```
use Illuminate\Support\Facades\Hash;
use App\Models\User;
```

Validating the Request

Add the following code to the `store` method:

```
public function store(Request $request): RedirectResponse
{

// Validate the incoming request data
$validatedData = $request->validate([
    'name' => 'required|string|max:255',
    'email' => 'required|string|email|max:255|unique:users',
    'password' => 'required|string|min:8|confirmed',
]);

print_r($validatedData);
die();
}
```

We are using the `validate` method to validate the incoming request data. If the validation fails, a redirect response will be generated to send the user back to their previous location. The errors will also be flashed to the session so they are available for display. The form should display the errors because of the way we handle the validation errors in the components.

As far as the rules, name must be a string, max 255 characters, email must be a string, email, max 255 characters, unique in the users table, password must be a string, min 8 characters, and it must be confirmed.

Hashing the Password

Before we save the user to the database, we need to hash the password. Laravel provides a helper function called `bcrypt` that we can use to hash the password. Update the `store` method in the `RegisterController` by adding the following line:

```
// Hash the password
$validatedData['password'] = Hash::make($validatedData['password']);
```

We are using the `Hash::make` method to hash the password. Now we can save the user to the database. We will use the `User` model to create a new user. Add the following line to the `store` method:

```
// Create a new user
$user = User::create($validatedData);
```

Redirecting the User

Let's redirect the user to the homepage after they register. Add the following line to the `store` method:

```
return redirect()->route('home')->with('success', 'Registration successful You can
now log in!');
```

Now you can register a new user. If you try to register a user with the same email address, you will get an error message. If you register a user successfully, you will be redirected to the homepage with a success message.

In the next lesson, we will add authentication to our application.

7 - User Authentication

Now that we can register a user, let's add log in functionality. We already have our `LoginController` and view. Let's add the form to the view.

Open the `resources/views/auth/login.blade.php` file and add the following code:

```
<x-layout>
  <div
    class="bg-white rounded-lg shadow-md w-full md:max-w-xl mx-auto mt-12 p-8 py-
12"
  >
    <h2 class="text-4xl text-center font-bold mb-4">Login</h2>
    <form method="POST" action="{{ route('login.authenticate') }}">
      @csrf
      <x-inputs.text
        id="email"
        name="email"
        type="email"
        placeholder="Email address"
      />
      <x-inputs.text
        id="password"
        name="password"
        type="password"
        placeholder="Password"
      />
      <button
        type="submit"
        class="w-full bg-blue-500 hover:bg-blue-600 text-white px-4 py-2 rounded
```

```
focus:outline-none"
      >
        Login
      </button>

      <p class="mt-4 text-gray-500">
        Don't have an account?
        <a class="text-blue-900" href="{{route('register')}}">Register</a>
      </p>
    </form>
  </div>
</x-layout>
```

This is very similar to the registration form we created earlier. The only difference is we don't have a name or confirm password fields. And obviously it is being submitted to a different route.

You should be able to go to `http://localhost:8000/login` and see the login page.

`authenticate` Method

Now let's handle the `authenticate` method in our `LoginController`. Open the `app/Http/Controllers/LoginController.php` file and add the following imports:

```
use Illuminate\Support\Facades\Auth;
```

Validate the Request Data

Add the following code to the method:

```
 public function authenticate(Request $request): RedirectResponse
{
    // Validate the request data
    $credentials = $request->validate([
        'email' => 'required|email',
        'password' => 'required|string',
    ]);

    dd($credentials);
}
```

You should see the credentials in the browser whether they are valid or not. Now we need to test if the credentials are valid.

Authenticate the User

Add the following code to the method:

```
 // Attempt to log the user in
if (Auth::attempt($credentials)) {
    // Regenerate the session to prevent fixation attacks
    $request->session()->regenerate();

    // Redirect to the intended route or a default route
    return redirect()->intended(route('home'))->with('success', 'You are now
logged in!');
}
```

This code will attempt to log in with the specified credentials. If the credentials are valid, it will log in the user and redirect them to the intended route or a default route.

The `regenerate()` method is used to regenerate the session to prevent fixation attacks. This is a security measure to prevent attackers from hijacking a user's session.

You should now be able to log in with the credentials you used to register. If you are not able to log in, check the credentials you used to register. If you are still having trouble, check the `User` model to make sure the password is being hashed.

When you log in, you will be redirected with a message. Check your `sessions` table and you should see an entry with `user_id` filled with your ID.

On Failure

If the credentials fail, we will redirect back with a message. Add this below the if statement you just wrote:

```
// If authentication fails, redirect back with an error message
       return back()->withErrors([
           'email' => 'The provided credentials do not match our records.',
       ])->onlyInput('email');
```

Next, we need to be able to log out. Let's add that functionality next.

8 - Log Out User & Auth Directive

Now that we are logged in, we need a way to log out. I also want to show and hide links based on the user's authentication status. We can use the `@auth` directive to do this.

Logout Route

Let's add a new route to the `routes/web.php` file:

```
Route::post('/logout', [LoginController::class, 'logout'])->name('logout');
```

`logout` Method

Next, let's add a `logout` method to the `LoginController` class:

```
public function logout(Request $request): RedirectResponse
{
    Auth::logout(); // Log out the user

    $request->session()->invalidate(); // Invalidate the session
    $request->session()->regenerateToken(); // Regenerate the CSRF token

    return redirect('/');
}
```

Logout Button & `@auth` Directive

Finally, let's add a logout button to the layout file. Open the `resources/views/components/header.blade.php` file and replace all the code with the following:

```
<header class="bg-blue-900 text-white p-4" x-data="{ open: false }">
  <div class="container mx-auto flex justify-between items-center">
    <h1 class="text-3xl font-semibold">
      <a href="{{ route('home') }}">Workopia</a>
    </h1>
    <nav class="hidden md:flex items-center space-x-4">
      <x-nav-link url="/" :active="request()->is('/')">Home</x-nav-link>
      <x-nav-link url="/jobs" :active="request()->is('jobs')"
        >All Jobs</x-nav-link
      >
      @auth
      <x-nav-link url="/jobs/saved" :active="request()->is('jobs/saved')"
        >Saved Jobs</x-nav-link
      >
      <x-nav-link
        url="/dashboard"
        :active="request()->is('dashboard')"
        icon="gauge"
        >Dashboard</x-nav-link
      >
      <form method="POST" action="{{ route('logout') }}">
        @csrf
        <button type="submit" class="text-white">
          <i class="fa fa-sign-out"></i> Logout
        </button>
      </form>
      <x-button-link url="/jobs/create" type="button" icon="edit"
        >Create Job</x-button-link
      >
      @else
      <x-nav-link url="/login" :active="request()->is('login')"
```

```
          >Login</x-nav-link
        >
        <x-nav-link url="/register" :active="request()->is('register')"
          >Register</x-nav-link
        >
        @endauth
      </nav>
      <button
        @click="open = !open"
        class="text-white md:hidden flex items-center"
      >
        <i class="fa fa-bars text-2xl"></i>
      </button>
    </div>
    <!-- Mobile Menu -->
    <nav
      x-show="open"
      @click.away="open = false"
      class="md:hidden bg-blue-900 text-white mt-5 pb-4 space-y-2"
    >
      <x-mobile-nav-link url="/" :active="request()->is('/')"
        >Home</x-mobile-nav-link
      >
      <x-mobile-nav-link url="/jobs" :active="request()->is('jobs')"
        >All Jobs</x-mobile-nav-link
      >
      @auth
      <x-mobile-nav-link url="/jobs/saved" :active="request()->is('jobs/saved')"
        >Saved Jobs</x-mobile-nav-link
      >
      <x-mobile-nav-link
        url="/dashboard"
        :active="request()->is('dashboard')"
        icon="gauge"
        >Dashboard</x-mobile-nav-link
      >
      <form method="POST" action="{{ route('logout') }}">
        @csrf
        <button type="submit" class="text-white">
          <i class="fa fa-sign-out"></i> Logout
        </button>
      </form>
      <div class="py-2">
        <x-button-link url="/jobs/create" type="button" icon="edit"
          >Create Job</x-button-link
        >
      </div>
      @else
      <x-mobile-nav-link url="/login" :active="request()->is('login')"
        >Login</x-mobile-nav-link
      >
      <x-mobile-nav-link url="/register" :active="request()->is('register')"
        >Register</x-mobile-nav-link
      >
```

```
    @endauth
  </nav>
</header>
```

You will only see certain links if you are logged in or logged out. The logout button will kill the session and redirect you to the home page.

Authenticate On Register

One more thing I want to do when it comes to our authentication system is to authenticate the user right after they register.

Open the `RegisterController` class and add the following import:

```
use Illuminate\Support\Facades\Auth;
```

Then in the `store` method, add the following line right above the `return redirect()->route('home')` line:

```
Auth::login($user);
```

Now the user will be logged in after they register.